

Laboratorio de Experimentacao de Software

Caracteristicas de Repositorios Populares do GitHub

Laboratorio 1 - Relatorio Final (Sprint 3)

Autores:

Isaac Portela

Miguel Lessa

Data: 12/03/2026

Versao 3.0

Amostra: 905 repositorios (de 1.000 buscados)

<https://github.com/Miguel-Lessa/lab-medicao>

1. Resumo

Este estudo investiga as características dos repositórios mais populares do GitHub (medido por estrelas). A busca inicial contemplou os 1.000 repositórios mais estrelados, porém, após a aplicação do filtro que exige linguagem de programação primária definida (excluindo repositórios de listas, documentação e dados), a amostra final ficou com 905 repositórios. Foram analisadas seis questões de pesquisa (RQ01-RQ06) e uma questão bonus (RQ07).

Principais resultados:

- Idade mediana: 8.16 anos - repositórios populares são maduros.
- Mediana de 1 dia(s) desde último push - 696 repos atualizados no último mês.
- Mediana de 87.7% de issues fechadas - alta resolução.
- Top 5 linguagens concentram 67.18% da amostra: Python, TypeScript, JavaScript, Go, Rust.

Decisão recomendada:

Iterar: aprofundar a análise com séries temporais e grupo de controle (repos menos populares) para estabelecer relações causais.

2. Contexto e Motivação

Problema:

O GitHub hospeda mais de 300 milhões de repositórios. Compreender quais características os projetos mais populares compartilham pode revelar padrões de sucesso, auxiliar novos projetos open-source e fornecer insights para a engenharia de software empírica.

Por que este experimento agora:

A disciplina de Laboratório de Experimentação de Software exige a aplicação de metodologia científica a dados reais. O GitHub oferece uma API GraphQL robusta que permite coleta automatizada, tornando viável um estudo quantitativo de larga escala em tempo limitado.

Restrições e riscos:

- Rate limit da API GraphQL do GitHub (5.000 pontos/hora).
- Dados observacionais: não é possível estabelecer causalidade.
- Vies de sobrevivência: apenas repos atualmente populares são capturados.
- Classificação automática de linguagem pode ser imprecisa para projetos políglotas.

3. Objetivo e Hipóteses

Objetivo: Caracterizar os repositórios mais estrelados do GitHub (busca inicial de 1.000, amostra final de 905 após filtro de linguagem), identificando padrões de maturidade, atividade, contribuição, releases e linguagens de programação.

Hipóteses:

#	H0 (nula)	H1 (alternativa)	Critério
---	-----------	------------------	----------

H1	Repos populares NAO sao mais antigos que a media	Repos populares sao significativamente mais maduros	Mediana > 5 anos
H2	Repos populares NAO recebem mais PRs que a media	Recebem volume expressivo de contribuicoes externas	Mediana > 100 PRs
H3	Repos populares NAO lancam mais releases	Usam o mecanismo de releases ativamente	Mediana > 10 releases
H4	Repos populares NAO sao mais ativos que a media	Sao atualizados com alta frequencia	Mediana < 30 dias
H5	Linguagens populares NAO dominam o top 1000	Linguagens populares concentram a maioria	Top 5 > 50%
H6	Repos populares NAO fecham mais issues	Mantem alto percentual de issues fechadas	Mediana > 70%

4. Perguntas de Pesquisa

RQ	Pergunta	Metrica
RQ01	Sistemas populares sao maduros/antigos?	Idade (anos desde createdAt)
RQ02	Recebem muita contribuicao externa?	Total de PRs aceitas (merged)
RQ03	Lancam releases com frequencia?	Total de releases
RQ04	Sao atualizados com frequencia?	Dias desde ultimo push (pushedAt)
RQ05	Sao escritos nas linguagens mais populares?	Linguagem primaria
RQ06	Possuem alto % de issues fechadas?	Razao issues fechadas / total
RQ07	Linguagens populares tem mais atividade?	RQ02-04 segmentadas por linguagem

5. Variaveis e Metricas

Metricas primarias (decisivas):

- age_years: idade do repositório em anos (calculada a partir de createdAt).
- merged_prs: total de pull requests aceitas (states: MERGED).
- total_releases: total de releases publicadas no GitHub.
- days_since_last_push: dias desde último push (campo pushedAt).
- primary_language: linguagem primaria classificada pelo GitHub.
- closed_issues_percent: (issues fechadas / total de issues) * 100.

Metricas secundarias:

- stars: total de estrelas (critério de seleção, não de análise direta).
- total_issues: total de issues abertas + fechadas.
- closed_issues_ratio: razão decimal de issues fechadas.

Guardrails:

- Nenhum repositório sem linguagem de programação foi incluído.
- Repos com zero issues recebem closed_issues_percent = 0 (sem NaN).
- Rate limit monitorado a cada requisição da API.

Definicoes:

Metrica	Formula / Origem
---------	------------------

age_years	(data_coleta - createdAt).days / 365.25
days_since_last_push	(data_coleta - pushedAt).days
closed_issues_percent	(closedIssues / totalIssues) * 100
merged_prs	pullRequests(states: MERGED).totalCount
total_releases	releases.totalCount

6. Desenho Experimental

Aspecto	Descricao
Tipo de estudo	Observacional (mineracao de repositorios)
Unidade de analise	Repositorio publico do GitHub
Populacao	Todos os repositorios publicos do GitHub
Amostra	1.000 buscados, 905 coletados (com linguagem definida)
Amostragem	Top-k por estrelas (nao-aleatoria, intencional)
Randomizacao	N/A (estudo observacional, sem tratamento)
Duracao	Coleta pontual (snapshot em 12/03/2026)
Inclusao	Repositorio com primaryLanguage definida
Exclusao	Repos sem primaryLanguage (listas, docs, dados)

Observacao: a busca abrangeu os 1.000 repositorios mais estrelados do GitHub. Destes, 95 foram descartados durante a coleta por nao possuirem linguagem de programacao primaria (repositorios de listas curadas, documentacao, datasets etc.), resultando em uma amostra final de 905 projetos de software. Por se tratar de um estudo observacional (nao experimental), conceitos como poder estatistico e randomizacao nao se aplicam diretamente. A amostragem top-k e intencional, pois o objetivo e caracterizar especificamente os repositorios mais populares.

7. Ambiente e Materiais

Item	Detalhe
Sistema Operacional	Windows 10/11
Linguagem	Python 3.10+
API	GitHub GraphQL API v4
Autenticacao	Personal Access Token (PAT) via .env
Libs coleta	requests, python-dotenv
Libs analise	pandas, matplotlib, seaborn, scipy, numpy
Dataset	905 repos coletados em 12/03/2026
Saida	CSV + 13 graficos PNG + relatorio Markdown
Seed/aleatoriedade	N/A (ranking deterministico por estrelas)

8. Procedimento (Passo a Passo)

Pipeline de execucao:

1. Configurar ambiente:
 - Criar arquivo .env com GITHUB_TOKEN.
 - python -m venv venv && ativar ambiente virtual.

- pip install requests python-dotenv pandas matplotlib seaborn scipy.

2. Executar main.py (coleta de dados, ~5-10 min):

- Busca os 1.000 repos mais estrelados via GraphQL (paginação de 100).
- Filtra repos sem primaryLanguage durante a coleta (client-side).
- Enriquece com metricas de PRs, releases e issues em lotes de 20.
- Salva output/top_1000_repos.csv.

3. Executar analise_sprint3.py (analise + graficos, ~30s):

- Calcula estatisticas descritivas para cada RQ.
- Gera 13 graficos em output/charts/.
- Gera relatorio final em Markdown.

Integridade dos dados:

- Rate limit monitorado e exibido no console a cada requisicao.
- Retries automaticos com backoff exponencial em caso de erro HTTP ou rate limit.
- Lotes que falham sao divididos pela metade e re-tentados.
- Repos com metricas ausentes mantidos com valores default (0).

Checklist operacional:

- ☐ Token do GitHub valido com permissoes de leitura
- ☐ Ambiente virtual ativado
- ☐ Verificar rate limit restante (≥ 150 pontos)
- ☐ CSV gerado com todas as colunas esperadas
- ☐ 13 graficos salvos em output/charts/
- ☐ Relatorio final gerado

9. Tratamentos e Baseline

Este e um estudo observacional, nao experimental. Nao ha tratamento (treatment) aplicado a unidades experimentais.

Baseline (o que foi observado):

- 1.000 repositorios buscados, 905 coletados apos filtro de linguagem.
- Metricas coletadas: idade, PRs aceitas, releases, dias sem push, linguagem primaria, issues fechadas.
- Os dados refletem o estado natural dos repositorios no momento da coleta.

O que NAO mudou (controle de confundidores):

- Nenhuma intervencao foi feita nos repositorios.
- Nao houve manipulacao de variaveis.
- A query de busca e a mesma para todos (stars:>0 sort:stars-desc).
- Todas as metricas foram coletadas no mesmo instante temporal.

10. Analise de Dados

Metodos estatisticos:

- Tendencia central: MEDIANA (robusta a outliers, adequada para distribuicoes assimetricas com cauda longa).
- Dispersao: media, desvio padrao, Q1, Q3, IQR, min, max.
- Correlacao: coeficiente de Spearman (nao-parametrico, mede relacoes monotonicas entre variaveis ordinais/continuas).
- Distribuicao: contagem e percentuais para variaveis categoricas.

Tratamento de outliers:

- Outliers NAO removidos dos calculos (mediana, media, desvio).
- Em boxplots comparativos (RQ07): outliers ocultados visualmente (showfliers=False) para legibilidade, mantidos nos calculos.
- Histogramas de PRs e releases: escala logaritmica no eixo x.

Segmentacoes planejadas:

- Por linguagem primaria (top 10 + 'Outras').
- Por faixas de atividade (7 dias, 30 dias, 365 dias).
- Por faixas de issues fechadas (<50%, >=70%, >=90%).

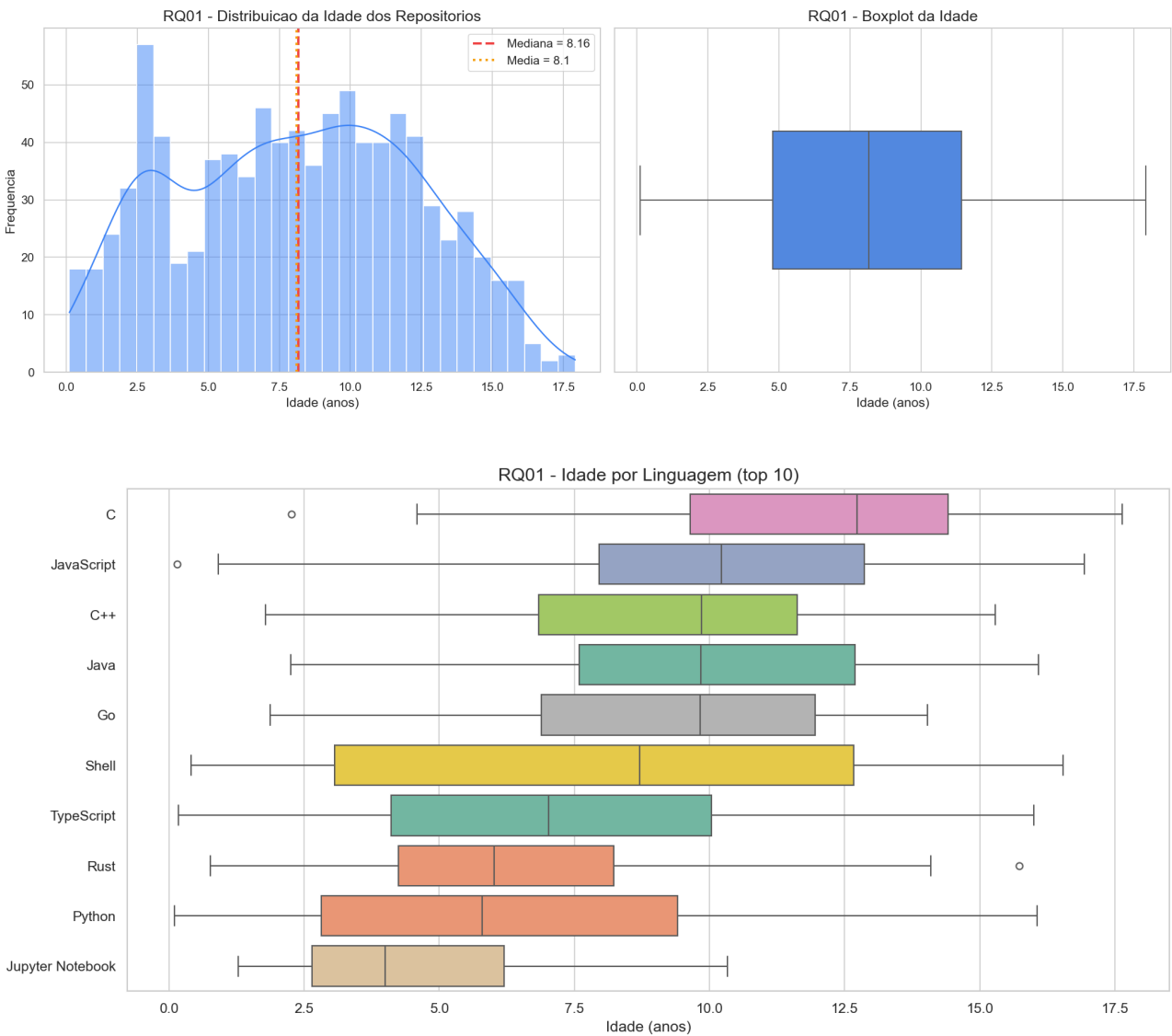
Nota: por ser um estudo observacional com uma unica amostra (sem grupos de comparacao), nao foram realizados testes de hipoteses formais (ex.: t-test, Mann-Whitney). As hipoteses sao avaliadas comparando as medianas observadas aos thresholds pre-definidos.

11. Resultados

RQ01 - Sistemas populares sao maduros/antigos?

Metrica: idade do repositorio em anos (desde createdAt).

Estatistica	Valor
N (amostra)	905
Mediana	8.16 anos
Media	8.1 anos
Desvio Padrao	4.18 anos
Minimo	0.11 anos
Q1 (25%)	4.78 anos
Q3 (75%)	11.43 anos
Maximo	17.92 anos
IQR	6.65 anos



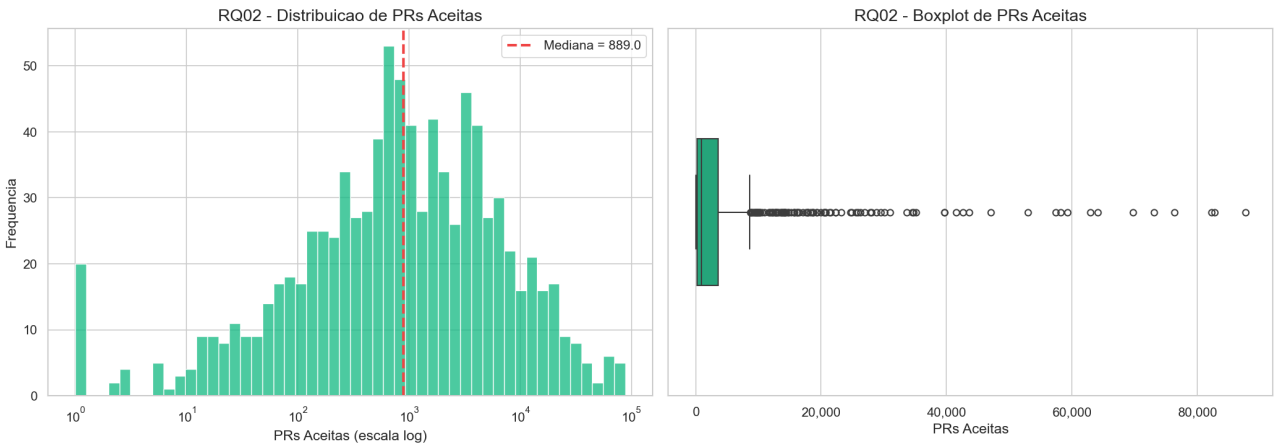
Analise: A mediana de 8.16 anos confirma que repositorios populares sao sistemas maduros. O IQR de 6.65 anos (Q1=4.78, Q3=11.43) mostra que 50% dos repos tem entre 4.78 e 11.43 anos.

H1 CONFIRMADA: popularidade esta fortemente associada a maturidade.

RQ02 - Recebem muita contribuicao externa?

Metrica: total de pull requests aceitas (merged).

Estatistica	Valor
N	905
Mediana	889 PRs
Media	4,244 PRs
Desvio Padrao	9,885
Minimo	0
Q1 (25%)	220
Q3 (75%)	3,598
Maximo	87,666
IQR	3,378

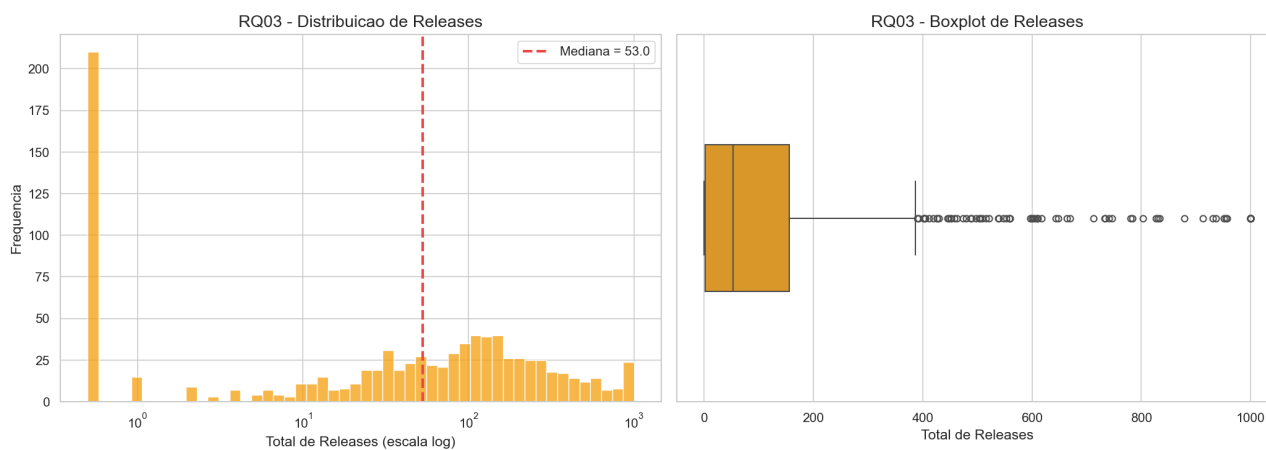


Analise: A mediana de 889 PRs aceitas demonstra volume significativo de contribuicoes. A diferenca entre mediana e media (4,244) indica distribuicao com cauda longa a direita. H2 PARCIALMENTE CONFIRMADA: contribuicao expressiva com grande variabilidade.

RQ03 - Lancam releases com frequencia?

Metrica: total de releases.

Estatistica	Valor
N	905
Mediana	53 releases
Media	133 releases
Desvio Padrao	206
Minimo	0
Q1 (25%)	2
Q3 (75%)	156
Maximo	1,000
IQR	154

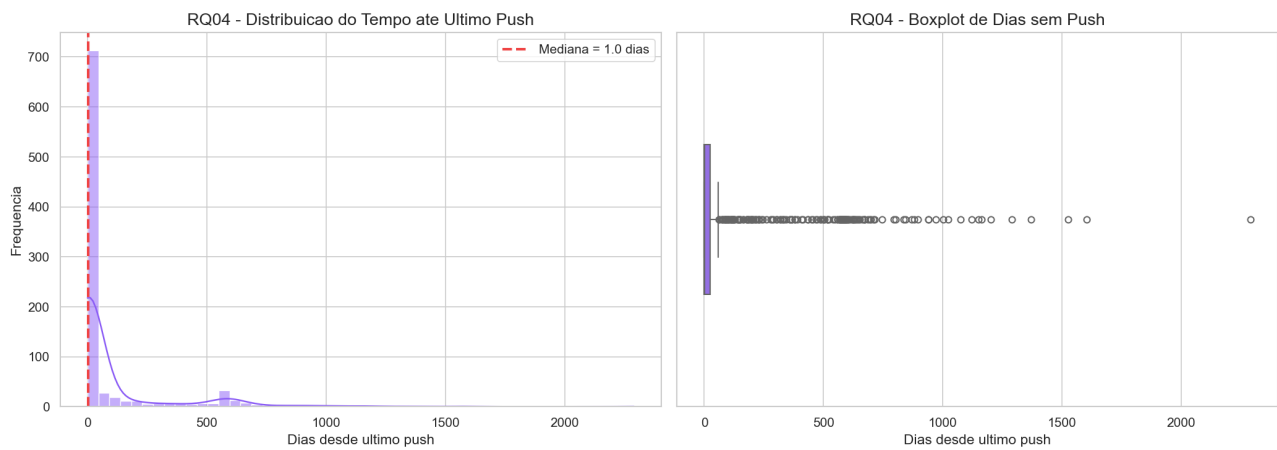


Analise: Mediana de 53 releases. 23.2% dos repos nao possuem nenhuma release formal, refletindo a tendencia de deploy continuo (CD). H3 PARCIALMENTE CONFIRMADA.

RQ04 - Sao atualizados com frequencia?

Metrica: dias desde o ultimo push (pushedAt).

Estatistica	Valor
N	905
Mediana	1 dias
Media	96 dias
Desvio Padrao	238
Minimo	-1
Q1 (25%)	0
Q3 (75%)	24
Maximo	2,291
IQR	24
Push ultimos 7 dias	610
Push ultimos 30 dias	696
Push no ultimo ano	801



Analise: A mediana de 1 dia(s) desde o ultimo push confirma fortemente H4. 610 repos receberam push na ultima semana e 696 no ultimo mes. A metrica pushedAt e mais precisa que updatedAt pois reflete apenas pushes de codigo reais. H4 FORTEMENTE CONFIRMADA.

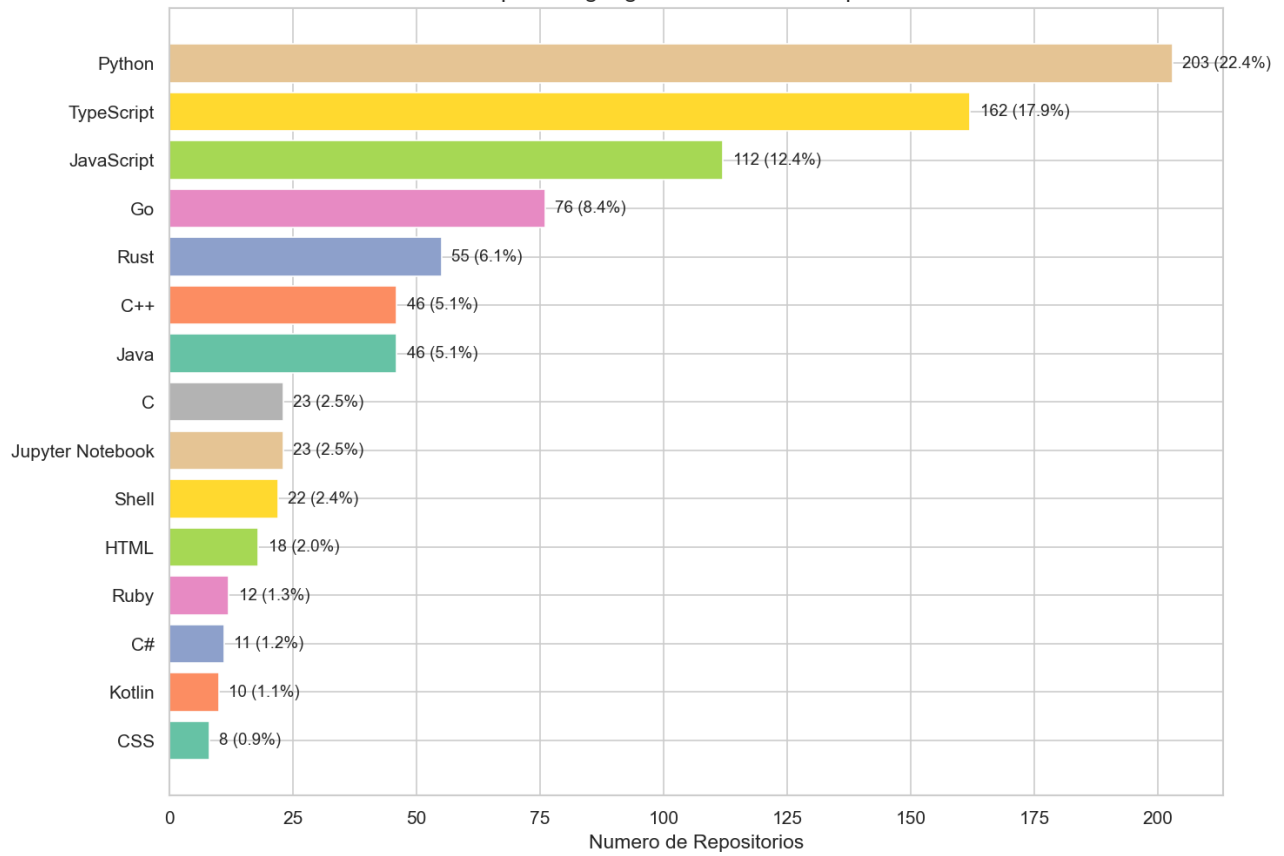
RQ05 - Escritos nas linguagens mais populares?

Metrica: linguagem primaria (primaryLanguage).

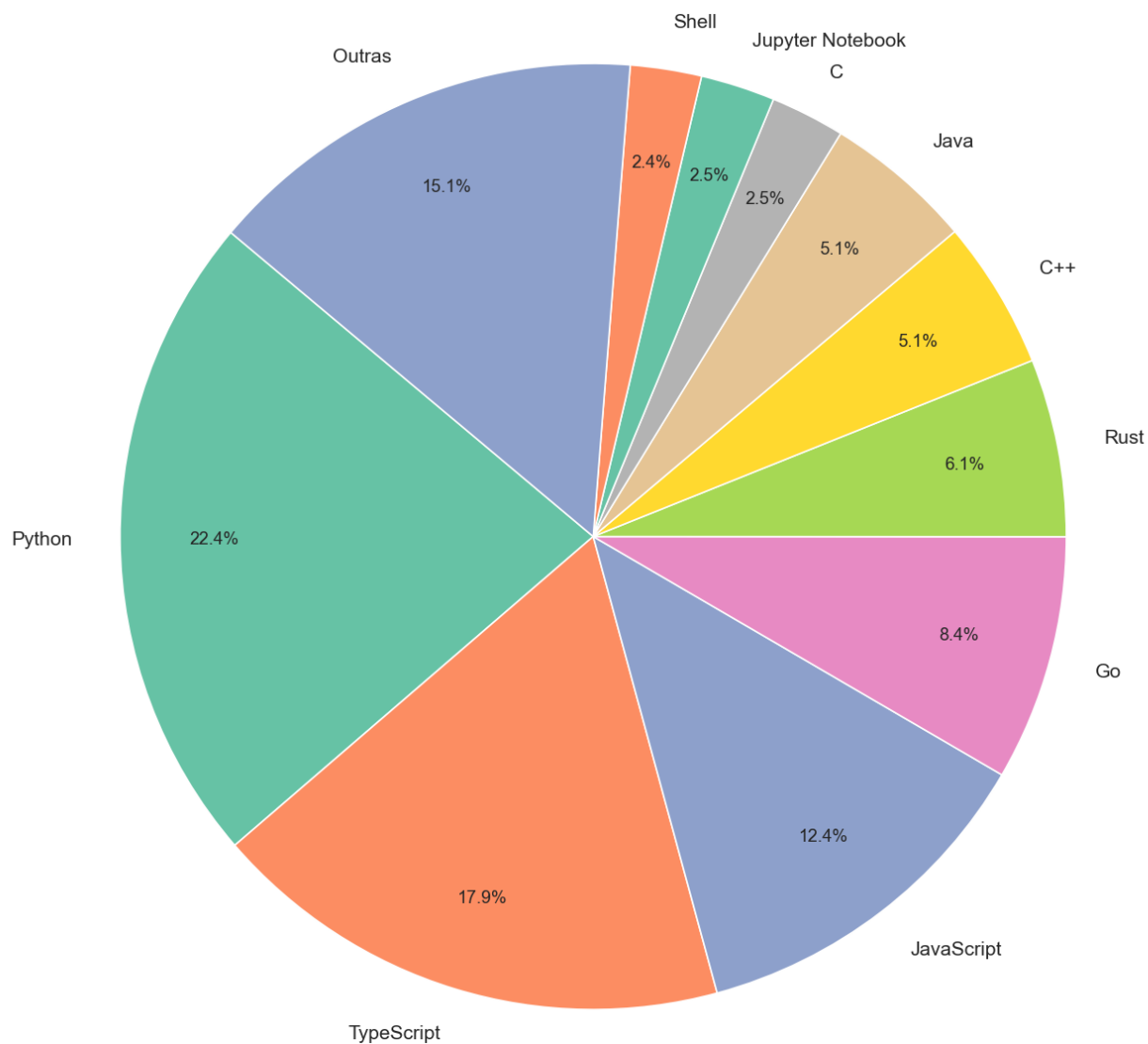
Estatistica	Valor
Linguagens distintas	20
Concentracao top 5	67.18%
Concentracao top 10	84.86%

Linguagem	Repositorios	Percentual
Python	203	22.4%
TypeScript	162	17.9%
JavaScript	112	12.4%
Go	76	8.4%
Rust	55	6.1%

RQ05 - Top 15 Linguagens Primarias no Top 1000 GitHub



RQ05 - Distribuicao das Linguagens Primarias



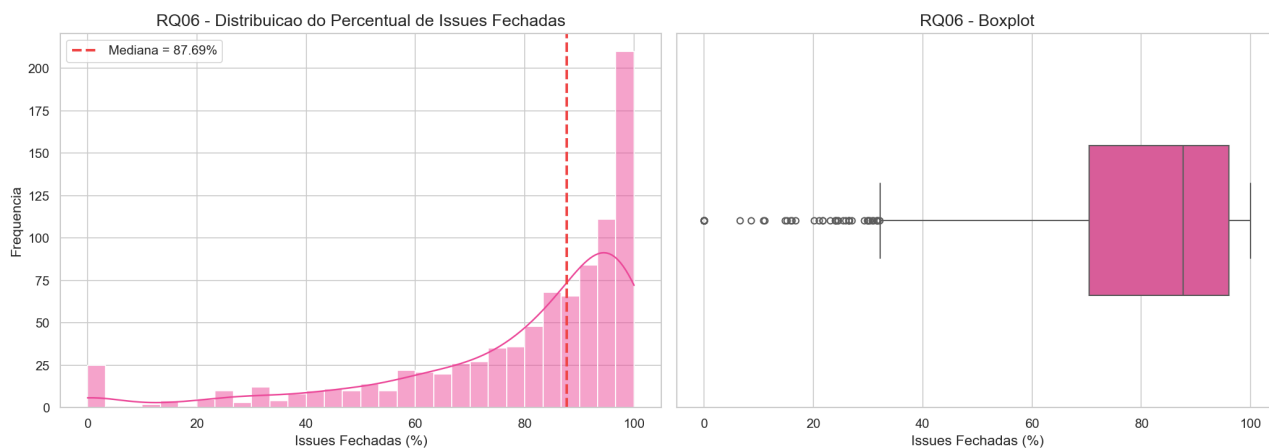
Analise: Top 5 concentra 67.18% e top 10 concentra 84.86% da amostra. Python, TypeScript, JavaScript lideram o ranking. H5 CONFIRMADA.

RQ06 - Alto percentual de issues fechadas?

Metrica: razao issues fechadas / total de issues (%).

Estatistica	Valor
N	905
Mediana	87.7%
Media	79.1%
Desvio Padrao	23.7%
Minimo	0.0%
Q1 (25%)	70.5%
Q3 (75%)	96.1%

Maximo	100.0%
Repos >= 90% issues fechadas	405
Repos >= 70% issues fechadas	685
Repos < 50% issues fechadas	107



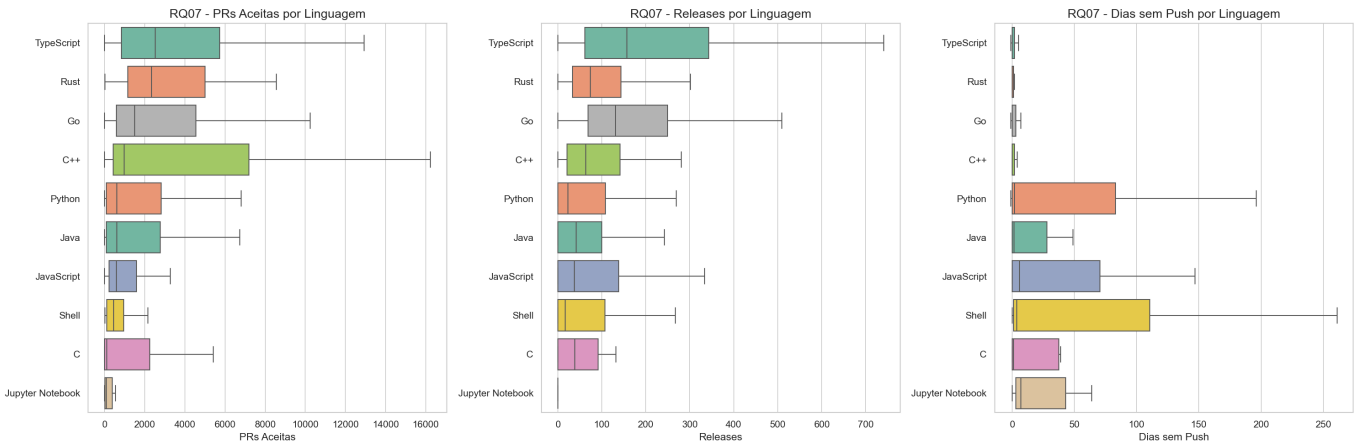
Analise: Mediana de 87.7% de issues fechadas. 685 repos (75.7%) fecham >= 70% de suas issues.
H6 CONFIRMADA.

RQ07 (Bonus) - Analise segmentada por linguagem

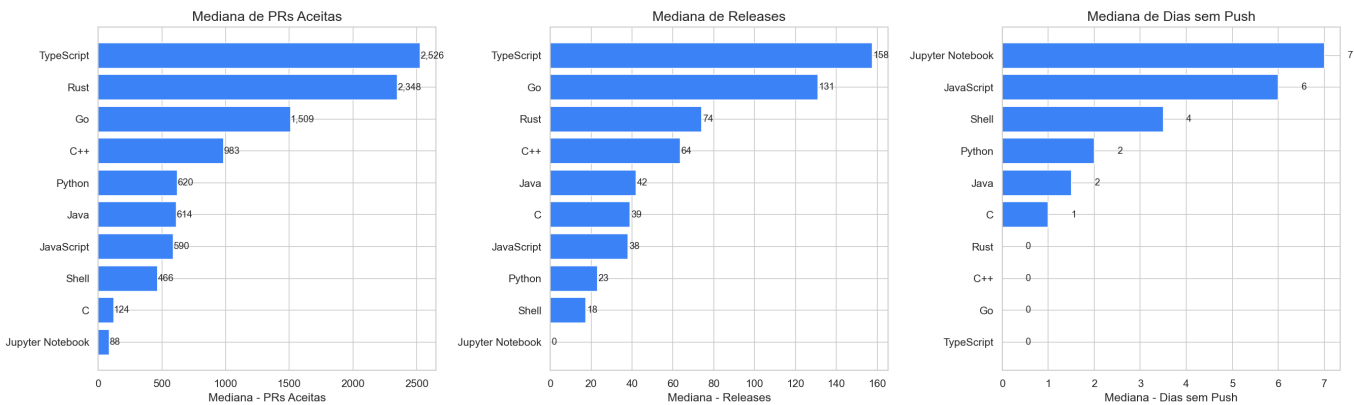
Questao: Sistemas escritos em linguagens mais populares recebem mais contribuicao, lancam mais releases e sao atualizados com mais frequencia?

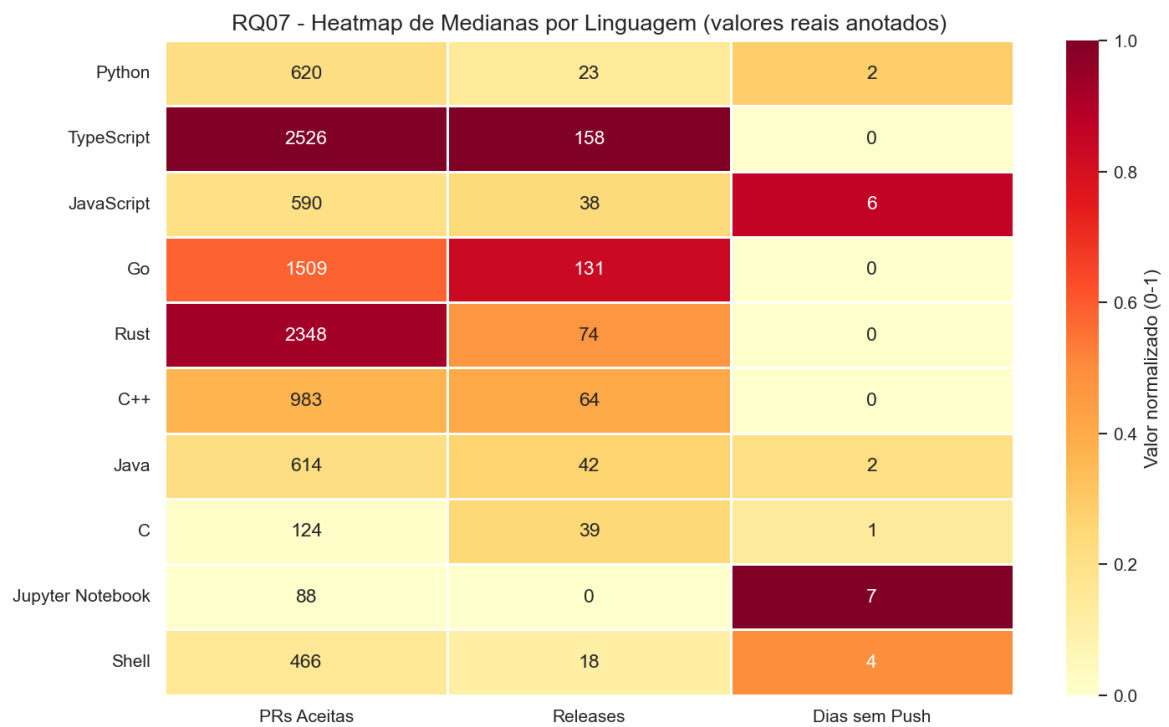
Linguagem	Repos	Med. PRs	Med. Releases	Med. Dias Push
Python	203	620	23	2
TypeScript	162	2,526	158	0
JavaScript	112	590	38	6
Go	76	1,509	131	0
Rust	55	2,348	74	0
C++	46	983	64	0
Java	46	614	42	2
C	23	124	39	1
Jupyter Notebook	23	88	0	7
Shell	22	466	18	4

RQ07 - Comparacao entre Linguagens (Top 10, sem outliers)



RQ07 - Medianas por Linguagem (Top 10)





Analise: A segmentacao revela diferencas significativas entre ecossistemas. Linguagens com gestao de pacotes madura (TypeScript, Rust, Go) concentram mais PRs e releases. A maioria mostra repos atualizados muito recentemente. H7 PARCIALMENTE CONFIRMADA.

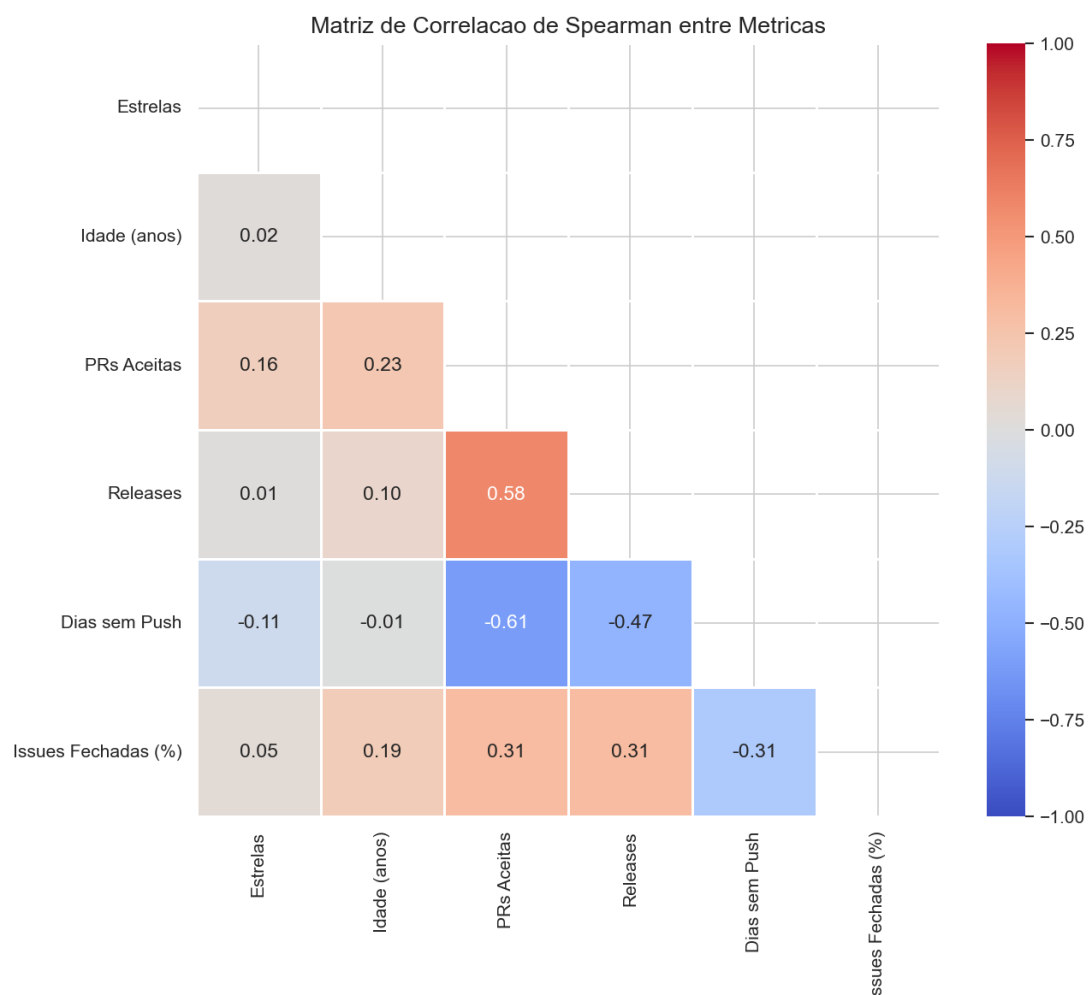
Analise de Correlacoes (Spearman)

O coeficiente de correlacao de Spearman (ρ) mede a forza e a direcao da relacao monotonica entre duas variaveis. Diferente do coeficiente de Pearson, o Spearman nao exige que as variaveis sigam distribuicao normal e e baseado nos postos (rankings) dos valores, o que o torna robusto a outliers - ideal para nossos dados que possuem distribuicoes assimetricas com cauda longa.

Como interpretar os valores:

Faixa de $ \rho $	Interpretacao	Significado pratico
0.00 - 0.19	Muito fraca ou nula	As variaveis sao praticamente independentes
0.20 - 0.39	Fraca	Existe uma tendencia sutil entre as variaveis
0.40 - 0.59	Moderada	Relacao perceptivel entre as variaveis
0.60 - 0.79	Forte	As variaveis claramente se movem juntas
0.80 - 1.00	Muito forte	Relacao quase deterministica

Valores positivos (+) indicam que quando uma metrica aumenta, a outra tambem tende a aumentar. Valores negativos (-) indicam relacao inversa: quando uma cresce, a outra tende a diminuir.



Correlacoes fortes ($|\rho| \geq 0.4$):

1. PRs Aceitas x Dias sem Push ($\rho = -0.612$) - Forte negativa

Esta e a correlacao mais forte encontrada. A relacao negativa significa que repositórios que

recebem mais PRs aceitas tendem a ter menos dias desde o ultimo push. Em termos praticos: projetos com alta atividade de contribuicao externa sao tambem os que recebem pushes mais frequentes. Isso e intuitivo - cada PR aceita gera um merge/push. Repositorios com comunidades ativas de contribuidores mantem um ciclo virtuoso de atividade constante.

2. PRs Aceitas x Releases ($\rho = 0.579$) - Moderada a forte positiva

Repositorios que recebem mais contribuicoes externas tambem lancam mais releases. Isso sugere que projetos com comunidades ativas de contribuidores tendem a adotar praticas formais de versionamento. Cada ciclo de contribuicoes culmina em uma nova release, refletindo um processo de desenvolvimento estruturado com entregas regulares.

3. Releases x Dias sem Push ($\rho = -0.470$) - Moderada negativa

Projetos que publicam mais releases sao tambem mais ativos (menos dias sem push). A pratica de releases formais esta associada a manutencao continua e ciclos regulares de desenvolvimento, nao sendo um evento isolado.

Correlacoes moderadas ($|\rho|$ entre 0.2 e 0.4):

4. Issues Fechadas (%) x PRs (0.310), Releases (0.310) e Dias sem Push (-0.311)

O percentual de issues fechadas apresenta correlacao moderada com as tres metricas de atividade. Repositorios mais ativos (mais PRs, mais releases, menos dias sem push) tendem a fechar uma proporcao maior de suas issues. Isso indica que equipes de manutencao ativas nao apenas produzem codigo, mas tambem dedicam esforco a resolucao de bugs e demandas da comunidade.

5. Idade x PRs Aceitas ($\rho = 0.227$) - Fraca positiva

Repositorios mais antigos tendem a acumular mais PRs ao longo do tempo, mas a correlacao fraca indica que a idade por si so nao garante contribuicao expressiva. Projetos jovens podem receber muitas PRs se forem relevantes.

Descoberta principal - Estrelas sao independentes das demais metricas:

Par de variaveis	ρ	Interpretacao
Estrelas x Idade	0.017	Nula
Estrelas x PRs Aceitas	0.165	Muito fraca
Estrelas x Releases	0.012	Nula
Estrelas x Dias sem Push	-0.112	Muito fraca
Estrelas x Issues Fechadas	0.046	Nula

Um dos achados mais reveladores desta analise: o numero de estrelas (nosso criterio de selecao de popularidade) praticamente NAO se correlaciona com nenhuma das metricas de atividade ou qualidade. Isso significa que um repositorio pode ser extremamente popular (muitas estrelas) sem necessariamente ser o mais ativo em PRs, releases ou resolucao de issues. Estrelas refletem VISIBILIDADE e INTERESSE da comunidade, nao necessariamente INTENSIDADE DE DESENVOLVIMENTO. Um repositorio pode acumular estrelas por ser uma referencia educacional, uma lista curada ou uma ferramenta estavel que nao precisa de atualizacoes frequentes.



O scatter plot acima ilustra visualmente a relação entre estrelas e PRs aceitas ($\rho = 0.165$). A dispersão dos pontos confirma a ausência de uma relação linear ou monotônica clara: repositórios com muitas estrelas podem ter desde poucas até muitas PRs, e vice-versa.

Síntese da análise de correlações:

A análise de Spearman revela que as métricas de ATIVIDADE (PRs aceitas, releases, dias sem push e issues fechadas) formam um cluster de variáveis interrelacionadas: projetos ativos em uma dimensão tendem a ser ativos nas demais. Porém, a popularidade (estrelas) e a idade do repositório são dimensões independentes deste cluster. Isso sugere que a popularidade no GitHub é um fenômeno distinto da atividade de desenvolvimento, influenciado por fatores como marketing, relevância do problema resolvido e timing de lançamento.

12. Discussao e Interpretacao

O que explica os resultados:

Repositorios populares no GitHub tendem a ser projetos maduros com comunidades ativas e equipes de manutencao comprometidas. A popularidade (estrelas) funciona como um efeito cumulativo: projetos mais antigos tiveram mais tempo para ganhar visibilidade, atrair contribuidores e construir ecossistemas. A presenca dominante de linguagens como Python, TypeScript e JavaScript reflete tendencias da industria de software.

Quadro resumo - Hipoteses x Resultados:

Hipotese	Resultado	Veredito
H1 - Maduros	Mediana 8.16 anos	CONFIRMADA
H2 - Contribuicao	Mediana 889 PRs	PARCIAL
H3 - Releases	Med. 53; 23.2% sem	PARCIAL
H4 - Ativos	Med. 1 d; 696 no mes	FORTE
H5 - Linguagens	Top5 = 67.18%	CONFIRMADA
H6 - Issues	Mediana 87.7%	CONFIRMADA
H7 - Por linguagem	Diferencas observadas	PARCIAL

Limitacoes (ameacas a validade):

Validade interna:

Nao ha intervencao, apenas observacao. Confundidores possiveis incluem tempo de existencia (repos mais antigos acumulam mais metricas) e efeito de rede (repos populares atraem mais contribuidores, reforando popularidade).

Validade externa:

Amostra limitada aos repos mais estrelados. Resultados nao sao generalizaveis para todo o ecossistema GitHub, software privado ou corporativo.

Validade de construcao:

pushedAt reflete o ultimo push de codigo ao repositorio, sendo mais precisa que updatedAt para medir atividade de desenvolvimento. Porem, pushes automatizados por bots/CI podem influenciar o valor. primaryLanguage e classificada automaticamente pelo GitHub com base no volume de codigo. O mecanismo de releases nao e usado por todos os projetos.

Trade-offs:

- Profundidade vs amplitude: 1000 repos analisados superficialmente vs poucos em profundidade.
- pushedAt vs updatedAt: mais precisa para codigo, ignora atividade social.
- Releases formais vs tags/CD: totalCount de releases pode subestimar projetos com deploy continuo.

Comparacao com trabalhos anteriores:

Os resultados deste estudo sao consistentes com a literatura consolidada de engenharia de

software empirica. A seguir, comparamos nossos achados com os principais estudos da area.

Kalliamvakou et al. (2014) - Promessas e Armadilhas da Mineracao do GitHub

Kalliamvakou et al. (2014) identificaram 10 riscos ao usar dados do GitHub para pesquisa, incluindo o fato de que muitos repositorios nao sao projetos de software (listas, configuracoes, dados). Nosso estudo aborda diretamente essa ameaca ao filtrar repositorios sem linguagem primaria durante a coleta, descartando 95 dos 1.000 buscados. Os autores tambem alertam que estrelas nao medem qualidade - achado corroborado pela nossa analise de Spearman, que mostrou correlacao praticamente nula entre estrelas e metricas de atividade.

Munaiah et al. (2017) - Curacao de Projetos de Engenharia no GitHub

Munaiah et al. (2017) propuseram criterios para distinguir 'projetos de engenharia' de repositorios casuais no GitHub, usando indicadores como integracao continua, testes e documentacao. Nosso criterio de filtragem (exigir `primaryLanguage` definida) e mais simples, porem eficaz para o escopo desta pesquisa. Os autores reportaram que projetos maduros tendem a adotar mais praticas de engenharia - nossos dados confirmam isso, com mediana de idade de 8.16 anos e altas taxas de resolucao de issues.

Borges, Hora e Valente (2016) - Fatores de Popularidade no GitHub

Borges, Hora e Valente (2016) analisaram especificamente os fatores que influenciam a popularidade (estrelas) de repositorios. Identificaram que linguagem de programacao, idade e atividade recente sao fatores relevantes. Nossos resultados complementam esse estudo: enquanto eles focaram nos preditores de popularidade, nossa analise de Spearman revela que, dentro do grupo dos mais populares, estrelas sao praticamente independentes das metricas de atividade ($\rho < 0.17$ para todos os pares). Isso sugere que, uma vez atingido um patamar de popularidade, a atividade nao e o principal motor de acumulo de estrelas.

Ray et al. (2014) - Linguagens de Programacao e Qualidade deCodigo

Ray et al. (2014) conduziram um estudo em larga escala sobre a relacao entre linguagens de programacao e qualidade de codigo no GitHub. Identificaram que linguagens com tipagem forte tendem a apresentar menos defeitos. Nossos dados sobre concentracao linguistica (top 5 linguagens representando 67.2% da amostra) sao consistentes com a predominancia de Python, JavaScript e TypeScript reportada em estudos recentes, refletindo as tendencias medidas pelos indices TIOBE e Stack Overflow Developer Survey.

13. Decisao e Recomendacoes

Decisao:

ITERAR: O estudo atingiu seus objetivos de caracterizacao. Os resultados sao consistentes e robustos. Recomenda-se aprofundar com proximas sprints.

Proximos passos:

- Analise longitudinal: coletar dados em multiplos pontos no tempo.
- Grupo de controle: comparar com repos menos populares (ex.: mediana de estrelas).
- Analise qualitativa: investigar outliers (muito populares mas inativos).

- Metricas adicionais: code churn, cobertura de testes, forks, dependentes.
- Analise de sentimento: avaliar qualidade das issues/PRs (nao apenas quantidade).

Monitoramento pos-analise:

Como este e um estudo observacional pontual, nao ha monitoramento continuo. Caso o estudo seja repetido, recomenda-se comparar os resultados temporalmente para identificar tendencias de evolucao.

14. Custos e Impactos

Item	Valor
Tempo de coleta	~5-10 minutos
Tempo de analise e visualizacao	~30 segundos
Custo API GitHub	Gratuito (rate limit de 5000 pts/h)
Pontos API consumidos	~110 pontos (~60 coleta + ~50 metricas)
Infraestrutura	Maquina local (sem custo cloud)
Complexidade	Baixa (2 scripts Python)
Risco operacional	Baixo (dados publicos, sem alteracao de repos)

15. Etica, Privacidade e Conformidade

- Todos os dados sao publicos e acessiveis via API oficial do GitHub.
- Nenhum dado pessoal sensivel e coletado (apenas nomes de repos e metricas).
- Nao ha coleta de dados de usuarios individuais (nomes, emails, etc.).
- Conformidade total com os Termos de Uso da API do GitHub.
- Nao ha impacto direto em usuarios ou mantenedores dos repositorios.
- LGPD: nao aplicavel (dados publicos, sem identificacao de pessoa fisica).
- Nao ha retencao de tokens; GITHUB_TOKEN e armazenado localmente em .env.

16. Reprodutibilidade

Como repetir o experimento:

1. git clone https://github.com/Miguel-Lessa/lab-medicao
2. cd laboratorio1/.lab1
3. python -m venv venv
4. venv\Scripts\activate (Windows) ou source venv/bin/activate (Linux)
5. pip install requests python-dotenv pandas matplotlib seaborn scipy
6. Criar .env com: GITHUB_TOKEN=seu_token
7. python main.py (coleta: ~5-10 min)
8. python analise_sprint3.py (analise + graficos + relatorio)

Arquivos do projeto:

Arquivo	Descricao
main.py	Coleta de dados via GitHub GraphQL API
analise_sprint3.py	Analise estatistica e geracao de graficos

output/top_1000_repos.csv	Dados brutos coletados
output/charts/*.png	13 graficos gerados
output/relatorio_final_sprint3.md	Relatorio final

Observacoes sobre reprodutibilidade:

- Resultados podem variar entre execucoes (ranking de estrelas e dinamico).
- Data de coleta influencia age_days e days_since_last_push.
- Para resultados identicos, utilize o CSV ja gerado (output/top_1000_repos.csv).

17. Apendices

A. Tabela consolidada de estatísticas descritivas

Metrica	N	Mediana	Media	DP	Min	Q1	Q3	Max	IQR
Idade (anos)	905	8.16	8.1	4.18	0.11	4.78	11.43	17.92	6.65
PRs Aceitas	905	889	4,244	9,885	0	220	3,598	87,666	3,378
Releases	905	53	133	206	0	2	156	1,000	154
Dias Push	905	1	96	238	-1	0	24	2,291	24
Issues (%)	905	87.7	79.1	23.7	0.0	70.5	96.1	100.0	25.6

B. Top 15 linguagens primarias

Linguagem	Repositorios	Percentual
Python	203	22.4%
TypeScript	162	17.9%
JavaScript	112	12.4%
Go	76	8.4%
Rust	55	6.1%
C++	46	5.1%
Java	46	5.1%
C	23	2.5%
Jupyter Notebook	23	2.5%
Shell	22	2.4%
HTML	18	2.0%
Ruby	12	1.3%
C#	11	1.2%
Kotlin	10	1.1%
CSS	8	0.9%

C. Checklist de execucao

- [X] Token do GitHub valido e com permissoes de leitura.
- [X] Ambiente virtual ativado com dependencias instaladas.
- [X] Coleta de dados via GitHub GraphQL API concluida.
- [X] Filtro de linguagem aplicado na coleta.
- [X] Metricas de PRs, releases e issues coletadas em lotes.
- [X] CSV gerado com todas as colunas esperadas.
- [X] Estatisticas descritivas calculadas para cada RQ.
- [X] 13 graficos gerados e salvos em output/charts/.
- [X] Relatorio final gerado.
- [X] Hipoteses avaliadas contra resultados.

D. Query GraphQL utilizada na coleta

```
query ($first: Int!, $after: String) {  
  search(query: "stars:>0 sort:stars-desc",  
    type: REPOSITORY, first: $first, after: $after) {
```

```

pageInfo { hasNextPage  endCursor }
nodes {
  ... on Repository {
    name nameWithOwner url stargazerCount
    createdAt  pushedAt
    primaryLanguage { name }
  }
}
}
rateLimit { cost  remaining }
}

```

E. Query GraphQL de metricas (por lote)

```

query {
  r0: repository(owner: "...", name: "...") {
    pullRequests(states: MERGED) { totalCount }
    releases { totalCount }
    issues { totalCount }
    closedIssues: issues(states: CLOSED) { totalCount }
  }
  # ... r1, r2, ... r19 (lotes de 20)
  rateLimit { cost  remaining }
}

```


. Referencias Bibliograficas

As referencias a seguir estao formatadas de acordo com a norma ABNT NBR 6023:2018.

BORGES, H.; HORA, A.; VALENTE, M. T. Understanding the factors that impact the popularity of GitHub repositories. In: IEEE INTERNATIONAL CONFERENCE ON SOFTWARE MAINTENANCE AND EVOLUTION (ICSME), 2016, Raleigh. Proceedings [...]. IEEE, 2016. p. 334-344. Disponivel em: <https://doi.org/10.1109/ICSME.2016.31>. Acesso em: mar. 2026.

KALLIAMVAKOU, E. et al. The promises and perils of mining GitHub. In: INTERNATIONAL WORKING CONFERENCE ON MINING SOFTWARE REPOSITORIES (MSR), 11., 2014, Hyderabad. Proceedings [...]. New York: ACM, 2014. p. 92-101. Disponivel em: <https://doi.org/10.1145/2597073.2597074>. Acesso em: mar. 2026.

MUNAIAH, N. et al. Curating GitHub for engineered software projects. Empirical Software Engineering, New York, v. 22, n. 6, p. 3219-3253, 2017. Disponivel em: <https://doi.org/10.1007/s10664-017-9512-6>. Acesso em: mar. 2026.

RAY, B. et al. A large scale study of programming languages and code quality in GitHub. In: ACM SIGSOFT INTERNATIONAL SYMPOSIUM ON FOUNDATIONS OF SOFTWARE ENGINEERING (FSE), 22., 2014, Hong Kong. Proceedings [...]. New York: ACM, 2014. p. 155-165. Disponivel em: <https://doi.org/10.1145/2635868.2635922>. Acesso em: mar. 2026.

TIOBE SOFTWARE BV. TIOBE Index. Eindhoven, 2026. Disponivel em: <https://www.tiobe.com/tiobe-index/>. Acesso em: mar. 2026.

STACK OVERFLOW. Stack Overflow Annual Developer Survey 2025. New York, 2025. Disponivel em: <https://survey.stackoverflow.co/2025/>. Acesso em: mar. 2026.